TypeScript

Generika a pokročilé koncepty

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

20.10.2026

CC BY-NC-SA 4.0

Úvod

Poslední díl TypeScriptu

Známe typový systém, třídy, rozhraní i SOLID principy

Jak ale napsat kód, který je **typově bezpečný** a přitom funguje s **různými typy**?

Dnes nás čeká:

- generika a utility typy
- dekorátory
- mixiny a composition over inheritance

Generika

Úvod do generik

Generika umožňují psát kód, který funguje s různými typy

Místo konkrétního typu použijeme **typový parametr**

Označuje se <T> nebo jiným písmenem

Úvod do generik

Proč generika?

Bez generik:

```
function identity(arg: number): number {
  return arg;
}
```

S generikami:

```
function identity<T>(arg: T): T {
  return arg;
}
```

Úvod do generik

Jedna funkce pro všechny typy!

Úvod do generik

Použití generické funkce:

```
const num = identity<number>(42);        // T=number
const str = identity<string>("hello");   // T=string

// Type inference
const num2 = identity(42);
// TypeScript inferred: T=number

const str2 = identity("hello");
// TypeScript inferred: T=string
```

Generické funkce

Generické funkce s polem:

```
function firstElement<T>(arr: T[]): T | undefined {
  return arr[0];
}

const numbers = [1, 2, 3];
const first = firstElement(numbers); // number | undefined

const strings = ["a", "b", "c"];
const firstStr = firstElement(strings); // string | undefined
```

Generické funkce

Kombinace typů donutí TypeScript inferovat nejlepší společný typ

```
const arr = [1, "2", 3];
const first = firstElement(arr); // T=(number | string)
```

Generické funkce s více typovými parametry

```
function pair<T, U>(first: T, second: U): [T, U] {
  return [first, second];
}

const p1 = pair("hello", 42);       // [string, number]
const p2 = pair(true, "world");     // [boolean, string]
```

Generické funkce s více typovými parametry

Praktický příklad - swap:

```
function swap<T, U>(tuple: [T, U]): [U, T] {
  return [tuple[1], tuple[0]];
}

const original: [string, number] = ["age", 30];
const swapped = swap(original); // [number, string]
console.log(swapped); // [30, "age"]
```

Generické třídy

Generické třídy fungují stejně jako generické funkce

```
class Box<T> {
  private content: T;

  constructor(value: T) {
    this.content = value;
  }

  getValue(): T {
    return this.content;
  }
```

Generické třídy

```
  setValue(value: T): void {
    this.content = value;
  }
}
```

Generické třídy

```
const numberBox = new Box<number>(123);
console.log(numberBox.getValue()); // 123
numberBox.setValue(456);

const stringBox = new Box<string>("hello");
console.log(stringBox.getValue()); // "hello"
```

Praktický příklad - generický Stack

```
class Stack<T> {
  private items: T[] = [];

  push(item: T): void {
    this.items.push(item);
  }

  pop(): T | undefined {
    return this.items.pop();
  }
```

Praktický příklad - generický Stack

```
  peek(): T | undefined {
    return this.items[this.items.length - 1];
  }
}
```

Praktický příklad - generický Stack

```
const numberStack = new Stack<number>();
numberStack.push(1);
numberStack.push(2);
console.log(numberStack.pop()); // 2

const stringStack = new Stack<string>();
stringStack.push("hello");
stringStack.push("world");
console.log(stringStack.pop()); // "world"
```

Generická omezení (constraints)

Constraints omezují, jaké typy můžeme použít

Používá se klíčové slovo `extends`

```
function getLength<
  T extends { length: number }
>(arg: T): number {
  return arg.length;
}
```

Generická omezení (constraints)

```
getLength("hello");   // OK - string has length
getLength([1, 2, 3]); // OK - array has length
getLength(123);       // Error - number doesn't have length
```

Generická omezení (constraints)

Constraint s rozhraním:

```
interface HasId {
  id: number;
}

function printId<T extends HasId>(obj: T): void {
  console.log(obj.id);
}

printId({ id: 1, name: "John" }); // OK
printId({ name: "Jane" });        // Error: missing 'id'
```

Generická omezení (constraints)

```
interface Nameable {
  name: string;
}

interface Ageable {
  age: number;
}
```

Generická omezení (constraints)

```
function printPerson<
  T extends Nameable & Ageable
>(person: T): void {
  console.log(`${person.name}, ${person.age} years old`);
}

printPerson({ name: "John", age: 30 }); // OK
printPerson({ name: "Jane" }); // Error: missing 'age'
```

Generická omezení (constraints)

Constraint pro klíče objektu:

```
function getProperty<
  T, K extends keyof T
>(obj: T, key: K): T[K] {
  return obj[key];
}
```

- `T` je typ objektu
- `K` je omezen na klíče objektu `T`

Neztrácíme typy při práci s vlastnostmi generického objektu

Generická omezení (constraints)

```
const person = { name: "John", age: 30 };
const name = getProperty(person, "name"); // string
const age = getProperty(person, "age");   // number
getProperty(person, "email"); // Error: "email" not in person
```

Utility typy

TypeScript obsahuje **vestavěné utility typy**

Usnadňují práci s typy bez nutnosti psát vlastní

Utility typy

Partial<T> - všechny vlastnosti volitelné

```
interface User {
  id: number;
  name: string;
  email: string;
}

function updateUser(user: User, updates: Partial<User>): User
{
  return { ...user, ...updates };
}
```

Utility typy

```
const user: User = {
  id: 1,
  name: "John",
  email: "john@example.com"
};

const updated = updateUser(user, { name: "Jane" }); // OK
```

Utility typy

Required - všechny vlastnosti povinné

```
interface Config {
  host?: string;
  port?: number;
}
function connect(config: Required<Config>): void {
  console.log(`Connecting to ${config.host}:${config.port}`);
}

connect({ host: "localhost", port: 3000 }); // OK
connect({ host: "localhost" }); // Error: missing 'port'
```

Utility typy

Readonly<T> - všechny vlastnosti readonly

```
interface Point {
  x: number;
  y: number;
}

const point: Readonly<Point> = { x: 10, y: 20 };
point.x = 5; // Error: Cannot assign to 'x'
```

Utility typy

Pick<T, K> - vyber pouze určité vlastnosti

```
interface User {
  id: number;
  name: string;
  email: string;
  password: string;
}

type PublicUser = Pick<User, "id" | "name" | "email">;
// { id: number; name: string; email: string; }
```

Utility typy

Omit<T, K> - vynech určité vlastnosti

```
type UserWithoutPassword = Omit<User, "password">;
// { id: number; name: string; email: string; }
```

Utility typy

Record<K, T> - vytvoř objekt s danými klíči a typem hodnot

```
type Role = "admin" | "user" | "guest";

const permissions: Record<Role, string[]> = {
  admin: ["read", "write", "delete"],
  user: ["read", "write"],
  guest: ["read"]
};
```

Pokročilé koncepty

Dekorátory

Dekorátor je speciální druh deklarace, kterou lze připojit k třídě, metodě, vlastnosti nebo parametru

Označuje se symbolem `@`

Umožňuje přidat metadata nebo změnit chování

Dekorátory

Pro použití dekorátorů v TypeScriptu <5.0 musíte povolit experimentální funkci v `tsconfig.json`:

```
{
  "compilerOptions": {
    "experimentalDecorators": true
  }
}
```

Nebo použít TypeScript 5.0+ s novými dekorátory (bez experimentalDecorators)

Typy dekorátorů

TypeScript podporuje několik typů dekorátorů:

1. **Class decorators** - pro třídy
2. **Method decorators** - pro metody
3. **Property decorators** - pro vlastnosti
4. **Parameter decorators** - pro parametry metod

Typy dekorátorů

Class decorator

```
function sealed(constructor: Function) {
  Object.seal(constructor);
  Object.seal(constructor.prototype);
}
```

`Object.seal` zabraňuje změnám na třídě a jejím prototypu

Typy dekorátorů

```
@sealed
class BugReport {
  type = "report";
  title: string;

  constructor(title: string) {
    this.title = title;
  }
}
```

Typy dekorátorů

Method decorator

```
function log(
  target: any,
  propertyKey: string,
  descriptor: PropertyDescriptor
) {
  const originalMethod = descriptor.value;
```

Typy dekorátorů

```
  descriptor.value = function(...args: any[]) {
    console.log(`Calling ${propertyKey} with`, args);
    const result = originalMethod.apply(this, args);
    console.log(`Result:`, result);
    return result;
  };
}
```

Typy dekorátorů

```
class Calculator {
  @log
  add(a: number, b: number): number {
    return a + b;
  }
}

const calc = new Calculator();
calc.add(2, 3);
// Calling add with [2, 3]
// Result: 5
```

Praktické příklady

Readonly decorator - znemožní změnu metody

```
function readonly(
  target: any,
  propertyKey: string,
  descriptor: PropertyDescriptor
) {
  descriptor.writable = false;
}
```

Praktické příklady

```
class Person {
  @readonly
  getName() {
    return "John";
  }
}
```

Pokud je funkce označena jako readonly, nemůže být přepsána

```
const person = new Person();
person.getName(); // "John"
person.getName = function() { return "Jane"; }; // Error
```

Praktické příklady

Validation decorator - validace parametrů

```
function validate(
  target: any,
  propertyKey: string,
  descriptor: PropertyDescriptor
) {
  const originalMethod = descriptor.value;
```

Praktické příklady

```
  descriptor.value = function(...args: any[]) {
    if (args.some(arg => arg < 0)) {
      throw new Error("Arguments must be positive");
    }
    return originalMethod.apply(this, args);
  };
}
```

Praktické příklady

```
class MathOperations {
  @validate
  divide(a: number, b: number): number {
    return a / b;
  }
}

const math = new MathOperations();
math.divide(10, 2);  // OK
math.divide(-10, 2); // Error: Arguments must be positive
```

Decorator factories

Decorator factory je funkce, která vrací dekorátor

Umožňuje předávat parametry do dekorátoru

```
function component(prefix: string) {
  return function(constructor: Function) {
    console.log(`${prefix}: ${constructor.name}`);
  };
}
```

Decorator factories

```
@component("APP")
class UserComponent {
  // ...
}
// Output: "APP: UserComponent"
```

Decorator factories

Parametrizovaný **method decorator**

```
function delay(milliseconds: number) {
  return function(
    target: any,
    propertyKey: string,
    descriptor: PropertyDescriptor
  ) {
    const originalMethod = descriptor.value;
```

Decorator factories

```
    descriptor.value = async function(...args: any[]) {
      await new Promise(
        resolve => setTimeout(resolve, milliseconds)
      );
      return originalMethod.apply(this, args);
    };
  };
}
```

Decorator factories

```
class API {
  @delay(1000)
  async fetchData() {
    return "Data loaded";
  }
}

const api = new API();
await api.fetchData(); // Wait 1s before calling
```

Decorator factories

Kombinace více parametrů

```
function logWithLevel(level: "info" | "warn" | "error") {
  return function(
    target: any,
    propertyKey: string,
    descriptor: PropertyDescriptor
  ) {
    const originalMethod = descriptor.value;
```

Decorator factories

```
    descriptor.value = function(...args: any[]) {
      console[level](
        `[${level.toUpperCase()}] ${propertyKey}`,
        args
      );
      return originalMethod.apply(this, args);
    };
  };
}
```

Decorator factories

```
class Service {
  @logWithLevel("info")
  start() { }

  @logWithLevel("error")
  handleError(err: Error) { }
}
```

Property decorators

Property decorator se aplikuje na vlastnosti třídy

Přijímá dva parametry:

- `target` - prototyp třídy (nebo konstruktor pro statické vlastnosti)
- `propertyKey` - název vlastnosti

```
function format(
  target: any,
  propertyKey: string
) {
  // Property decorator
}
```

Property decorators

Příklad: **uppercase** vlastnost

```
function uppercase(target: any, propertyKey: string) {
  let value: string;

  const getter = function() {
    return value;
  };
```

Property decorators

```
  const setter = function(newVal: string) {
    value = newVal.toUpperCase();
  };

  Object.defineProperty(target, propertyKey, {
    get: getter,
    set: setter,
    enumerable: true,
    configurable: true
  });
}
```

Property decorators

```
class User {
  @uppercase
  name: string;

  constructor(name: string) {
    this.name = name;
  }
}

const user = new User("john");
console.log(user.name); // "JOHN"
```

Property decorators

Required field validator

```
const requiredFields: Map<any, string[]> = new Map();

function required(target: any, propertyKey: string) {
  const constructor = target.constructor;
  const fields = requiredFields.get(constructor) || [];
  fields.push(propertyKey);
  requiredFields.set(constructor, fields);
}
```

Property decorators

```
function validate(instance: any): boolean {
  const fields = requiredFields.get(
    instance.constructor
  ) || [];

  return fields.every(field => {
    return instance[field] !== undefined
      && instance[field] !== null;
  });
}
```

Property decorators

```
class Form {
  @required
  username: string;

  @required
  email: string;

  age?: number;
}
```

Property decorators

```
const form = new Form();
form.username = "john";
validate(form); // false - email is missing
```

Dekorátory v praxi

Reálné frameworky TypeScriptu intenzivně využívají dekorátory

- **NestJS** - backend framework
- **TypeORM** - ORM knihovna
- **Angular** - frontend framework
- **MobX** - state management

Dekorátory v praxi

NestJS - HTTP controller

```
import { Controller, Get, Post, Body } from '@nestjs/common';

@Controller('users')
export class UsersController {
  @Get()
  findAll() {
    return 'This returns all users';
  }
```

Dekorátory v praxi

```
  @Get(':id')
  findOne(@Param('id') id: string) {
    return `This returns user #${id}`;
  }

  @Post()
  create(@Body() createUserDto: CreateUserDto) {
    return 'This creates a new user';
  }
}
```

Dekorátory v praxi

NestJS - Dependency Injection

```
import { Injectable } from '@nestjs/common';

@Injectable()
export class UsersService {
  private users = [];

  findAll() {
    return this.users;
  }
```

Dekorátory v praxi

```
  create(user: any) {
    this.users.push(user);
  }
}
```

Dekorátory v praxi

```
@Controller('users')
export class UsersController {
  constructor(
    private usersService: UsersService
  ) {}

  @Get()
  findAll() {
    return this.usersService.findAll();
  }
}
```

Dekorátory v praxi

TypeORM - Entity definice

```
// Import decorators from TypeORM
import {
  Entity,
  PrimaryGeneratedColumn,
  Column
} from 'typeorm';
```

Dekorátory v praxi

```
@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  firstName: string;

  @Column()
  lastName: string;
```

Dekorátory v praxi

```
  @Column({ unique: true })
  email: string;

  @Column({ default: true })
  isActive: boolean;
}
```

Dekorátory v praxi

TypeORM - Vztahy mezi entitami

```
import {
  Entity,
  Column,
  OneToMany,
  ManyToOne
} from 'typeorm';
```

Dekorátory v praxi

```
@Entity()
export class Post {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  title: string;

  @ManyToOne(() => User, user => user.posts)
  author: User;
}
```

Dekorátory v praxi

```
@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @OneToMany(() => Post, post => post.author)
  posts: Post[];
}
```

Dekorátory v praxi

TypeORM - Validace a transformace

```
import {
  Entity,
  Column,
  BeforeInsert
} from 'typeorm';
import * as bcrypt from 'bcrypt';
```

Dekorátory v praxi

```
@Entity()
export class User {
  @Column()
  password: string;
```

Dekorátory v praxi

```
  @BeforeInsert()
  async hashPassword() {
    this.password = await bcrypt.hash(
      this.password,
      10
    );
  }
}
```

Dekorátor `@BeforeInsert()` zajistí, že heslo bude **zahashováno** před uložením entity do databáze

Mixiny

Co jsou mixiny?

Mixin je vzor, který umožňuje kombinovat chování z více tříd

TypeScript nepodporuje vícenásobnou dědičnost, ale mixiny jsou alternativou

Mixiny přidávají funkcionalitu do třídy bez použití dědičnosti

Co jsou mixiny?

Proč používat mixiny?

- **Kombinace funkcionality** z více zdrojů
- **Znovupoužitelnost** kódu
- Obejití omezení **jednoduché dědičnosti**
- **Kompozice** místo dědičnosti

Implementace mixinů v TypeScriptu

Základní pattern pro mixiny:

```
// Mixin type
type Constructor<T = {}> = new (...args: any[]) => T;
```

Implementace mixinů v TypeScriptu

```
// Mixin function
function Timestamped<
  TBase extends Constructor
>(Base: TBase) {
  return class extends Base {
    timestamp = new Date();
    getTimestamp() {
      return this.timestamp;
    }
  };
}
```

Implementace mixinů v TypeScriptu

```
// Base class
class User {
  constructor(public name: string) {}
}

// Apply mixin
const TimestampedUser = Timestamped(User);

const user = new TimestampedUser("John");
console.log(user.name);                // "John"
console.log(user.getTimestamp()); // current date
```

Implementace mixinů v TypeScriptu

```
function Activatable<
  TBase extends Constructor
>(Base: TBase) {
  return class extends Base {
    isActive = false;
    activate() { this.isActive = true; }
    deactivate() { this.isActive = false; }
  };
}
```

Implementace mixinů v TypeScriptu

```
// Combining multiple mixins
class User {
  constructor(public name: string) {}
}

const EnhancedUser = Timestamped(Activatable(User));
```

Implementace mixinů v TypeScriptu

```
const user = new EnhancedUser("John");
console.log(user.name);           // "John"
console.log(user.getTimestamp()); // current date
user.activate();
console.log(user.isActive);       // true
```

Disposable pattern

```
function Disposable<
  TBase extends Constructor
>(Base: TBase) {
  return class extends Base {
    isDisposed = false;
    dispose() { this.isDisposed = true; }
  };
}
```

Disposable pattern

```
class Resource {
  constructor(public id: number) {}
}

const DisposableResource = Disposable(Resource);
```

Composition over Inheritance

Prefer composition over inheritance je důležitý princip v OOP

Co to znamená?

Místo dědičnosti („je“) preferujeme kompozici („má“)

Composition over Inheritance

Problémy s dědičností:

- **Těsné vazby** - potomci závisí na rodiči
- **Křehkost** - změna v rodiči může rozbít potomky
- **Omezení** - můžeme dědit jen z jedné třídy
- **Neflexibilita** - složité změnit hierarchii později

Composition over Inheritance

❌ **Špatně** - nadměrné použití dědičnosti:

```
class Vehicle {
  move() { console.log("Moving..."); }
}
class FlyingVehicle extends Vehicle {
  fly() { console.log("Flying..."); }
}
class SwimmingVehicle extends Vehicle {
  swim() { console.log("Swimming..."); }
}
```

Composition over Inheritance

Co ale s futuristickou lodí, která umí plout i létat?

Composition over Inheritance

✅ **Dobře** - použití kompozice:

```
interface Movement {
  execute(): void;
}
```

Composition over Inheritance

```
class FlyMovement implements Movement {
  execute() { console.log("Flying..."); }
}

class SwimMovement implements Movement {
  execute() { console.log("Swimming..."); }
}

class DriveMovement implements Movement {
  execute() { console.log("Driving..."); }
}
```

Composition over Inheritance

```
class Vehicle {
  constructor(
    private movements: Movement[]
  ) {}

  move() {
    this.movements.forEach(m => m.execute());
  }
}
```

Composition over Inheritance

```
// Flexible functionality composition
const airplane = new Vehicle([
  new FlyMovement(),
]);

const amphibiousVehicle = new Vehicle([
  new DriveMovement(),
  new SwimMovement()
]);
```

Composition over Inheritance

Výhody kompozice:

- **Flexibilita** - snadno měníme chování
- **Znovupoužitelnost** - komponenty lze kombinovat
- **Loose coupling** - volnější vazby
- **Testovatelnost** - snadnější testování jednotlivých částí

Composition over Inheritance

Praktický příklad - UI komponenty:

```
interface Plugin {
  apply(): void;
}
```

Composition over Inheritance

```
class Editor {
  private plugins: Plugin[] = [];

  addPlugin(plugin: Plugin) {
    this.plugins.push(plugin);
  }

  initialize() {
    this.plugins.forEach(p => p.apply());
  }
}
```

Composition over Inheritance

```
class SpellCheckPlugin implements Plugin {
  apply() { console.log("Spell check enabled"); }
}

class AutoSavePlugin implements Plugin {
  apply() { console.log("Auto-save enabled"); }
}
```

Composition over Inheritance

```
const editor = new Editor();
editor.addPlugin(new SpellCheckPlugin());
editor.addPlugin(new AutoSavePlugin());
editor.initialize();
```

Můžeme snadno přidávat/odebírat funkcionality!

Composition over Inheritance

Kdy použít dědičnost vs. kompozici?

Použijte dědičnost když:

- Máte jasný vztah „je“ (Dog **je** Animal)
- Sdílíte společný kód mezi příbuznými třídami
- Potřebujete polymorfismus

Použijte kompozici když:

- Máte vztah „má“ nebo „umí“
- Chcete flexibilně kombinovat funkce nebo často měnit funkcionalitu

Závěr

Shrnutí

Pokročilé koncepty

- Generika pro znovupoužitelný kód
- Utility typy (Partial, Required, Pick, …)
- Dekorátory pro metadata
- Mixiny pro kompozici
- Composition over Inheritance

Shrnutí

Klíčové poznatky

- TypeScript přidává **typovou bezpečnost** do JavaScriptu
- OOP principy pomáhají **organizovat** a **škálovat** kód
- **SOLID principy** vedou k lepšímu návrhu tříd a rozhraní
- Generika umožňují psát **flexibilní** a **znovupoužitelný** kód
- **Composition over inheritance** pro větší flexibilitu
- Dekorátory a mixiny jsou **pokročilé vzory** pro rozšíření funkcionality

Užitečné zdroje

Oficiální dokumentace

- TypeScript Handbook:
  https://www.typescriptlang.org/docs/
- TypeScript Playground (online editor):
  https://www.typescriptlang.org/play
- TypeScript GitHub:
  https://github.com/microsoft/TypeScript

Užitečné zdroje

Doporučená četba

- Effective TypeScript (Dan Vanderkam)
- Programming TypeScript (Boris Cherny)
- TypeScript Deep Dive
  https://basarat.gitbook.io/typescript/

Užitečné zdroje

Nástroje a ekosystém

- TSLint / ESLint - lintování kódu
- Prettier - formátování kódu
- ts-node - spouštění TypeScript bez kompilace
- DefinitelyTyped - typové definice pro JavaScript knihovny

Užitečné zdroje

Best practices

- Používejte **strict mode** v tsconfig.json
- Preferujte **interface** nad type alias pro objekty
- Používejte **readonly** kde je to možné
- Vyhněte se `any`, používejte `unknown`
- Dokumentujte kód pomocí JSDoc komentářů